Introduction to Python

AI Specialization · Python Pathway

Naeemullah Khan

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

LMH
Lady Margaret Hall
University of Oxford

KAUST Academy
King Abdullah University of Science and Technology
Stage 1 · Course 1

June 12, 2026

Module 1
FOUNDATIONS OF PROGRAMMING

Programming, computers, Python, and AI/ML foundations

Module Content

In this module, we build the foundation needed to understand programming, Python tools, notebooks, algorithms, and first Python output programs.

- What programming languages are, and why computers need precise instructions.
- Natural, high-level, low-level, and machine language; high-level vs. low-level comparison.
- Why programming matters in science, engineering, data science, and AI/ML.
- How computers execute instructions: input, program, CPU, memory, storage, and output.
- Why Python is popular, what it is good for, and how Python code runs.
- File types and workflows: `.py` scripts, bytecode, REPL, and notebooks.
- Programming environments: interpreter, code editor or IDE, terminal, and notebook environment.
- Using Google Colab for most course coding labs and explanations.
- First Python program: `print()`, strings, numbers, `sep`, `end`, comments, and docstrings.
- Algorithm thinking: correctness, efficiency, clarity, flowcharts, and pseudocode.
- Beginner examples: maximum value, searching, sorting, login validation, and Big-O intuition.
- Activities: printing patterns, self-introduction program, formatted receipt, and common mistakes.

What Is a Programming Language?

Imagine that you want to give your computer a task. You say:

"Hey computer, calculate 5 + 5."

The problem is that the computer does not understand natural human language in the same way we do. The instruction is ambiguous for the machine, so nothing useful happens.

Instead, we write instructions in a language the computer can follow.

Natural language request
"calculate 5 + 5"

× not precise enough for the computer

Python instruction
`print(5 + 5)`

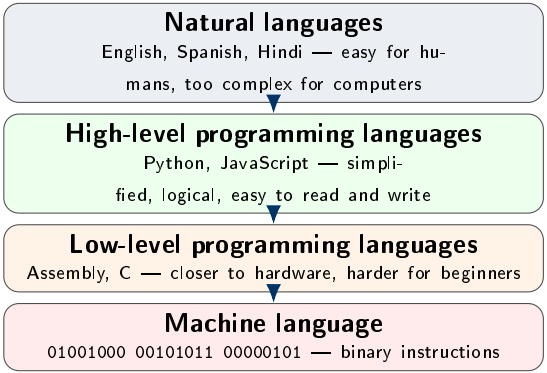
▼

Computer output
10

Key idea: **Program** = instructions. **Programming** = designing and implementing them.

From Human Language to Machine Language

Not all languages are the same. Some are made for people, while others are made for machines.



How to read the levels

Each step **down** brings us closer to how the machine thinks.

Each step **up** brings us closer to how humans think.

Python is a high-level bridge: it hides low-level complexity so we can focus on solving the problem.

High-Level vs. Low-Level Programming Languages

Feature	High-level languages	Low-level languages
Main idea	Closer to human reasoning and problem solving	Closer to how the hardware actually works
Examples	Python, JavaScript, Java, MATLAB	Assembly, C, machine code
Ease of reading	Easier to read, write, and learn	Harder to read, write, and debug
Control over hardware	Less direct hardware control; many details are hidden	More direct control over memory, registers, and hardware behavior
Development speed	Faster for building applications, experiments, and prototypes	Slower because the programmer manages more details
Performance focus	Often optimized through libraries and frameworks	Can be highly optimized for speed, memory, or embedded systems
Typical uses	Data science, AI/ML, web apps, automation, education, scientific computing	Operating systems, device drivers, embedded systems, microcontrollers, performance-critical code

Remember

High-level languages help us express ideas quickly. Low-level languages give more control over the machine, but require more technical detail.

Why Programming Matters in Science, Engineering, and AI

Programming allows us to:

- automate repeated tasks,
- analyze large datasets,
- simulate physical systems,
- build models that learn from data,
- visualize results,
- control robots, sensors, and experiments,
- create reliable and reproducible workflows.

AI/ML connection

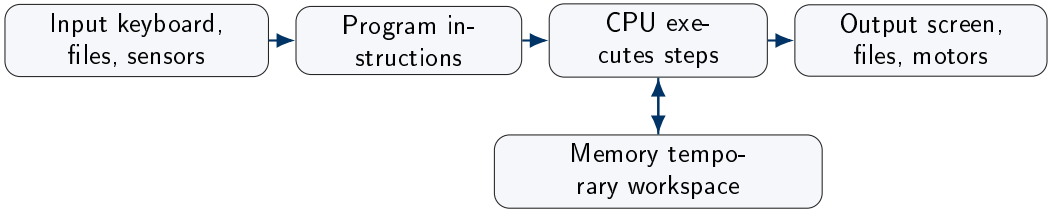
Machine learning is not magic. It is a programmed pipeline: load data, clean data, train a model, evaluate results, improve the process, and deploy the solution.

A Simple AI/ML Pipeline Is a Program

Each box is made of many smaller programming tasks:

- reading files, checking errors, filtering rows,
- computing statistics, using libraries,
- saving results, reporting metrics,
- repeating experiments consistently.

How Computers Execute Instructions: The Big Picture



- The **CPU** performs operations.
- **Memory** stores temporary values while the program runs.
- **Storage** keeps files and programs after power is off.
- **Input/output** connects the program to the outside world.

CPU, Memory, Storage: A Beginner Analogy

Computer part	What it does	Analogy
CPU	Executes instructions step by step	The chef cooking the recipe
Memory/RAM	Holds temporary data while the program runs	The kitchen counter
Storage	Holds files and programs long term	The pantry or recipe book shelf
Program	The instructions to follow	The recipe
Input	Data given to the program	Ingredients/orders
Output	Result produced by the program	Finished meal/receipt

Why Python Dominates Data Science and AI

Python is popular in data science and AI because it is:

- readable and beginner-friendly,
- expressive: fewer lines can do more work,
- supported by a huge scientific ecosystem,
- compatible with fast libraries written in C, C++, and Fortran,
- widely used in education, research, startups, and industry.

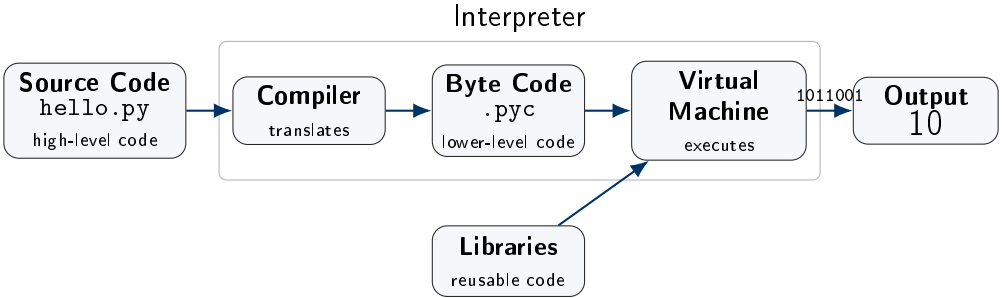
Important libraries you will meet later

`NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, PyTorch, TensorFlow, FastAPI, Jupyter`

What Python Is Good For

- Learning programming fundamentals
- Data analysis
- Automation scripts
- Artificial intelligence and machine learning
- Web APIs
- Scientific computing
- Prototyping ideas quickly
- Reading and writing files
- Visualizing data
- Working with databases
- Connecting to sensors and devices
- Building educational tools

How Python Works



- We write Python as **high-level source code** in a file such as `hello.py`.
- Python translates the script into **byte code**, often stored as a `.pyc` file.
- The **Python Virtual Machine** executes the byte code and calls libraries when needed.
- The final result is the program **output** that the user sees.

Python Script vs. Byte Code

Python script: human-readable source code

```
# file name: hello.py
print("Hello from Python")
print(5 + 5)
```

- Written by the programmer.
- Easy to read, edit, and share.
- Saved with the extension `.py`.

Byte code: machine-oriented instructions

```
64 00 5a 00 02 00 65 01
64 01 a6 01 ab 01 01 00
64 02 64 03 7a 00 00 00
```

- Generated by Python automatically.
- Not meant to be written by beginners.
- Often stored in `__pycache__` as `.pyc` files.

Key idea

Students write `.py` files. Python turns them into lower-level byte code so the interpreter can execute them efficiently.

Before We Write Python: The Missing Context

Key idea

Beginners should not only ask "What code do I write?" They should also ask "Where does this code live, and how does it run?"

A Python instruction can be written in different places:

- inside a saved **script file**,
- directly into an **interactive prompt**,
- inside a **notebook cell**,
- through an **IDE** or cloud environment.

Checkpoint

The same Python language can be used through different tools. The tool changes the workflow, not the core language.

Two Important File Types: .py and .ipynb

File type	What it contains	Typical use
.py	Plain Python code saved as a script	Programs, assignments, automation, reusable project files
.ipynb	Notebook document containing code cells, text cells, outputs, images, and plots	Data science, AI experiments, teaching, reports, exploration

Checkpoint

Both can contain Python code, but they support different learning and working styles.

Python Script Files: The .py Workflow

A **script** is a saved Python file that can be run again later.

```
# file name: hello.py
print("Hello from a Python script!")
print("This program can be saved and reused.")
```

Script workflow

1. Write code in a file such as `hello.py`.
2. Save the file.
3. Run the file using a terminal or an IDE run button.
4. Read the output and fix mistakes if needed.

Running a Script from the Terminal

If the file is named `hello.py`, we can run it from a terminal:

```
python hello.py
```

On some systems, the command may be:

```
python3 hello.py
```

Beginner interpretation

This command means: "Ask the Python interpreter to execute the file named `hello.py`."

Checkpoint

If the terminal says Python was not found, the issue is usually installation or system path configuration, not your program logic.

When Script Files Are Best

Scripts are best when the goal is to create something **repeatable**.

- A homework program that should run the same way tomorrow.
- A data-cleaning script used on many files.
- A small automation task.
- A project with multiple Python files.
- Code that will be shared through GitHub or submitted for review.

Important habit

A script should be readable from top to bottom. Someone else should be able to open the file and understand the order of execution.

Interactive Python: The REPL

Key idea

The REPL is an interactive Python environment where you type one instruction and immediately see the result.

REPL stands for:

- **Read:** Python reads what you typed.
- **Evaluate:** Python computes or executes it.
- **Print:** Python displays the result if there is one.
- **Loop:** Python waits for the next instruction.

Checkpoint

The REPL is excellent for quick experiments, but it is not ideal for long programs because the work is not naturally organized as a saved file.

REPL Example: Immediate Feedback

A REPL session often shows the prompt »>.

```
>>> 2 + 3
5
>>> print("Hello from the REPL")
Hello from the REPL
>>> "AI" + " " + "Python"
'AI Python'
```

What students should notice

The REPL is useful for testing tiny ideas before placing them into a larger script or notebook.

REPL vs. Script: Same Python, Different Workflow

Mode	Strength	Limitation
REPL	Fast feedback; great for tiny experiments	Not ideal for saving a complete program
Script	Organized, reusable, shareable	Slightly slower feedback because you save and run the file

Teaching analogy

The REPL is like trying one sentence aloud. A script is like writing a complete paragraph that can be read again later.

Interactive Python Notebook: Code + Explanation + Output

An **interactive Python notebook** is an `.ipynb` document made of cells.

Code cells

Contain Python code and show output directly below the cell.

Markdown cells

Contain formatted text, headings, equations, images, and explanations.

This makes notebooks useful for learning, experiments, and AI reports.

Why Notebooks Are Popular in AI and Data Science

Notebooks are common in AI/ML because they support an exploratory workflow:

1. Load data.
2. Inspect a few rows.
3. Clean or transform the data.
4. Plot results.
5. Train or test a model.
6. Explain observations in text.

AI/ML connection

A notebook can act like a lab notebook: it records not only the code, but also the reasoning, outputs, charts, and intermediate findings.

Notebook Execution Order: A Common Beginner Trap

Key idea

Notebook cells can be run independently, so the order you run them matters.

A notebook may look like a document from top to bottom, but the computer remembers what has already been executed.

- Running cell 5 before cell 2 may cause errors.
- Changing an earlier cell does not automatically rerun later cells.
- Old values can remain in memory and confuse results.

Checkpoint

Good notebook habit: when confused, restart the runtime/kernel and run all cells from top to bottom.

Programming Environment: What Do You Need?

To write and run Python, you need:

1. **Python interpreter**: the program that executes Python code.
2. **Code editor or IDE**: where you write code.
3. **Terminal/command line**: where you can run commands.
4. Optional: **Notebook environment**: useful for experiments, notes, and data analysis.

Recommended beginner setup

Install Python 3, then use VS Code or PyCharm. For data science notebooks, use Jupyter or Google Colab.

Common Tools

Tool	Purpose
Python.org installer	Installs the Python interpreter on your computer.
VS Code	Lightweight editor with Python extensions.
PyCharm	Full Python IDE with strong project tools.
Terminal	Runs Python scripts and installation commands.
REPL	Interactive Python prompt for quick experiments.
Notebook (.ipynb)	Mixes code, text, output, and plots in one document.
Google Colab	Cloud notebook environment; useful when local setup is difficult.

Checking That Python Works

In a terminal, try:

```
python --version
```

or, on some systems:

```
python3 --version
```

Expected output looks like:

```
Python 3.12.4
```

Checkpoint

If your terminal cannot find Python, the installation path may not be configured.

Class demonstration

We will demonstrate Python installation locally, running Python with an IDE, and using the interactive Python REPL.

Course Coding Labs: We Will Use Google Colab

Most of the course coding labs and explanations will be done in **interactive Python notebooks** using **Google Colab**.

This is useful because students can focus on learning Python concepts instead of spending too much time on setup problems.

- Code, explanation, and output can live in the same notebook.
- Notebooks are easy to share with a link.
- Colab runs in the browser and does not require a full local setup.
- It is suitable for beginner Python, data science, and later AI/ML work.

Main message

Colab will be our main learning environment, while local Python and IDEs are still important professional tools.

Google Colab: Cloud Notebooks for Beginners

Google Colab is a notebook environment that runs in the browser.

- No local Python installation is required.
- Code runs on Google's cloud machines.
- Notebooks are easy to share with a link.
- It supports GPUs and TPUs for later AI workloads.

Why this matters in Module 1

Students can begin learning Python immediately without losing time to installation problems.

What Actually Happens in Colab?

When you run a cell in Colab:

1. Your browser sends the code to a cloud runtime.
2. The runtime executes the Python code.
3. The output is sent back and displayed under the cell.

Runtime

A runtime is the temporary machine/session where your code executes. If the runtime disconnects or restarts, variables in memory are lost.

Checkpoint

Files and notebooks are not the same as memory. A saved notebook can remain, while runtime variables disappear after restart.

Why Colab Is Especially Useful for AI/ML

Colab fits AI/ML education because it supports later course needs:

- datasets can be uploaded or mounted from Google Drive,
- plots and tables appear directly in the notebook,
- GPU acceleration can be enabled for deep learning,
- notebooks can be shared with instructors or teammates,
- examples are easy to reproduce in class.

Beginner focus

In the first weeks, Colab is mainly used to remove setup friction. Later, it becomes useful for experiments and model training.

Recommended Learning Path for This Course

1. Start with **Google Colab** to avoid setup barriers.
2. Use notebooks to learn concepts, run examples, and document thinking.
3. Practice small `.py` scripts once basic syntax becomes familiar.
4. Move toward **VS Code** for larger projects and professional workflows.

Main principle

The goal is not to memorize tools. The goal is to understand where code runs and choose the tool that fits the task.

Class demonstration

We will demonstrate Google Colab, then write our first Python code using Colab.

Notebooks

A notebook contains cells.

- **Code cells**: contain Python code and output.
- **Markdown cells**: contain formatted explanation, equations, images, and notes.

Why notebooks are common in AI/ML

They combine experiment, explanation, visualization, and results in one place.

Caution

Notebook execution order matters. Running cells out of order can confuse beginners.

Module 2

First Programs, Syntax & Algorithmic Thinking

Output programs, syntax rules, and algorithmic thinking

Module Content

In this module, we will build our first Python programs and develop algorithmic thinking.

- **First program**: `print()`, strings, numbers, `sep`, `end`.
- **Comments and docstrings**: single-line and multi-line.
- **Python syntax**: indentation, colons, case sensitivity.
- **Algorithms**: correctness, efficiency, clarity.
- **Flowcharts**: symbols, drawing decision flows.
- **Pseudocode**: writing logic without syntax pressure.
- **Small programs**: self-introduction, formatted receipt.
- **Big-O intuition**: a first look at algorithm cost.

Learning goal

Plan before you code: think in algorithms, express in flowcharts or pseudocode, then translate to Python.

Your First Python Program

The classic first program prints a message:

```
print("Hello, World!")
```

What happens?

1. Python reads the instruction.
2. It recognizes `print` as a built-in function.
3. It sends the text `"Hello, World!"` to the screen.

Understanding print()

`print()` displays output.

```
print("Python is fun")
print(42)
print("The answer is", 42)
```

Vocabulary

- `print`: function name.
- Parentheses `()`: hold information given to the function.
- `"Python is fun"`: a string, meaning text data.
- `42`: a number.

The sep Parameter

By default, `print()` separates multiple values with a space.

```
print("A", "B", "C")
print("A", "B", "C", sep="-")
print("2026", "05", "03", sep="/")
```

Possible output:

```
A B C
A-B-C
2026/05/03
```

`sep` means separator.

The end Parameter

By default, `print()` ends with a new line.

```
print("Hello")
print("World")
```

Output:

```
Hello
World
```

Change the ending:

```
print("Hello", end=" ")
print("World")
```

Output:

```
Hello World
```

Comments

Comments are notes for humans. Python ignores them when running the program.

```
# This program prints a greeting
print("Welcome to Python") # Display message to user
```

Good comments explain why something is done, not only what the code already says.

Weak comment

```
print("Hello") # prints Hello
```

Better comment

```
# First output confirms that the environment is working
```

Docstrings

A docstring is a special string used to document a module, function, class, or method.
You will understand functions more deeply later, but here is an early preview:

```
def greet():
    """Display a simple greeting message."""
    print("Hello!")
```

Beginner takeaway

Comments and docstrings help people understand code. Good programmers write for both computers and humans.

Python Syntax: Tiny Details Matter

Computers are strict. Small syntax details can change meaning or cause errors.

Correct

```
print("Hello")
```

Incorrect

```
print("Hello"
```

Missing closing parenthesis.

Checkpoint

Syntax is the grammar of a programming language. Logic is the meaning of the steps. Both matter.

What Is an Algorithm?

Key idea

An algorithm is a finite, ordered set of steps for solving a problem.

A good algorithm should be:

- **Correct**: it solves the intended problem.
- **Clear**: each step is understandable.
- **Finite**: it eventually stops.
- **Unambiguous**: every step has one clear meaning.
- **Efficient enough**: it does not waste unnecessary time or memory.

Algorithm Example: Make Tea

1. Put water in kettle.
2. Turn kettle on.
3. Wait until water boils.
4. Put tea bag in cup.
5. Pour hot water into cup.
6. Wait 3 minutes.
7. Remove tea bag.
8. Add sugar if desired.
9. Serve tea.

Programming lesson

Even ordinary tasks have sequence, decisions, and repeated checks.

Correctness, Efficiency, and Clarity

Quality	Question to ask	Example
Correctness	Does it solve the right problem?	Does the login algorithm reject wrong passwords?
Efficiency	How much time and memory does it use?	Searching 1 million records should not be painfully slow.
Clarity	Can a human understand and maintain it?	Clear step names help others fix or improve the program.

Flowcharts: Visual Algorithms

Key idea

A flowchart represents the logic of an algorithm using shapes and arrows.

Flowcharts are useful because they:

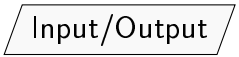
- show the order of steps,
- make decisions visible,
- reveal loops,
- help beginners understand logic before syntax,
- support communication between technical and non-technical people.

Common Flowchart Symbols

Beginning or ending point

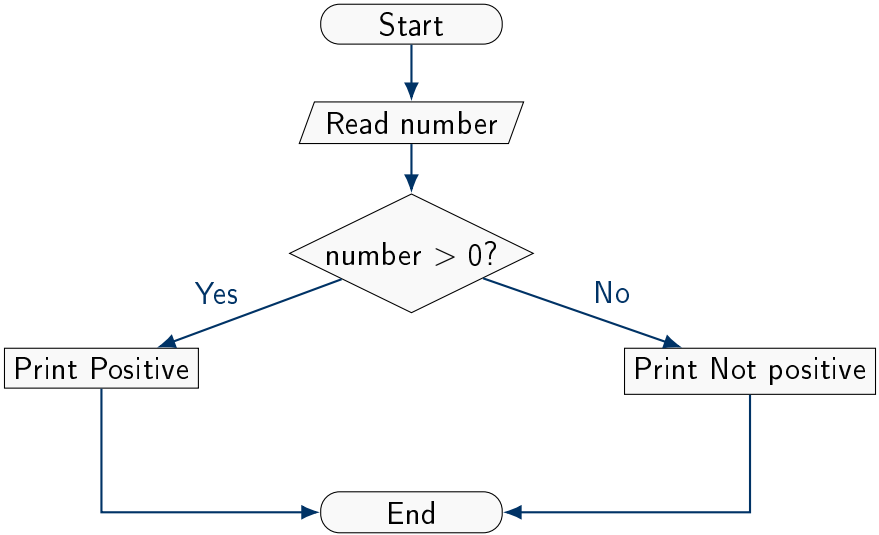
Action or calculation

Question with branches, usually Yes/No



Read input or display output

Flowchart Example: Is a Number Positive?



Pseudocode: Code-Like Thinking Without Syntax Pressure

Key idea

Pseudocode is an informal way to write algorithm steps using human-readable statements.

Pseudocode is not real Python. It is a planning tool.

Why use pseudocode?

- It focuses on logic, not punctuation.
- It can be understood by people who know different languages.
- It helps detect missing steps before coding.
- It makes translation into Python easier.

Pseudocode Example: Positive Number

```
START
READ number
IF number is greater than 0 THEN
     DISPLAY “Positive”
ELSE
     DISPLAY “Not positive”
END IF
END
```

Notice

The pseudocode is structured like code, but it is not tied to any exact programming language syntax.

Flowchart vs. Pseudocode vs. Python

Representation	Best for	Limitation
Flowchart	Visualizing decisions, branches, loops	Can become large for complex programs
Pseudocode	Planning logic in readable steps	Cannot be executed by computer
Python code	Running the actual program	Requires correct syntax

A beginner-friendly workflow:

$$\text{Problem} \rightarrow \text{Algorithm} \rightarrow \text{Flowchart} \rightarrow \text{Pseudocode} \rightarrow \text{Python}$$

Activity: Login Validation Flowchart

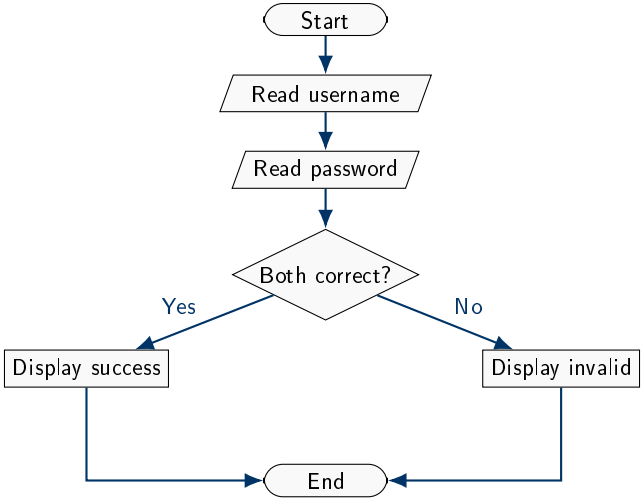
Task: Draw a flowchart for this process.

1. Start.
2. Ask for username.
3. Ask for password.
4. If username equals the stored username and password equals the stored password, display "Login successful."
5. Otherwise, display "Invalid username or password."
6. End.

Challenge

Add a maximum of 3 attempts. What changes in the flowchart?

Login Validation Flowchart



Algorithm Example: Find the Maximum Number

Problem: Given a list of numbers, find the largest value.

```
START
SET largest to first number in list
FOR each remaining number in list
    IF number is greater than largest THEN
        SET largest to number
    END IF
END FOR
DISPLAY largest
END
```

Important assumption

The list must contain at least one number. If the list is empty, there is no maximum.

Trace Example: Find Maximum

Input list: `[4, 9, 2, 11, 5]`

Step	Current number	Largest before	Action
Start	4	none	Set largest = 4
1	9	4	9 is greater than 4, set largest = 9
2	2	9	2 is not greater than 9, keep largest = 9
3	11	9	11 is greater than 9, set largest = 11
4	5	11	5 is not greater than 11, keep largest = 11

Final answer: `11`

Algorithm Example: Search for an Item

Problem: Determine whether a target item appears in a list.

```
START
SET found to False
FOR each item in list
    IF item equals target THEN
        SET found to True
        STOP searching
    END IF
END FOR
IF found is True THEN
    DISPLAY "Item found"
ELSE
    DISPLAY "Item not found"
END IF
END
```

Algorithm Example: Sorting Numbers Conceptually

Problem: Put numbers in increasing order.
One beginner-friendly idea: repeatedly select the smallest remaining item.

```
START
CREATE empty sorted list
WHILE original list is not empty
    FIND smallest number in original list
    REMOVE smallest number from original list
    ADD smallest number to sorted list
END WHILE
DISPLAY sorted list
END
```

This is clear, but not always the fastest method for large lists.

Introduction to Big-O: A First Intuition

Key idea

Big-O notation describes how the work of an algorithm grows as the input size grows.

At this stage, do not memorize formulas. Focus on the idea of growth.

Name	Informal meaning	Example intuition
$O(1)$	Constant time	Check the first item only
$O(n)$	Linear time	Check every item once
$O(n^2)$	Quadratic time	For every item, compare with every other item
$O(\log n)$	Logarithmic time	Repeatedly cut the problem in half

Big-O Example: Searching a List

Imagine searching for a name in a list.

Unsorted list

You may need to check every name.
Growth pattern: roughly $O(n)$.

Sorted list with smart search

You can check the middle, discard half, and repeat.
Growth pattern: roughly $O(\log n)$.

Checkpoint

Efficiency matters more when data becomes large, which is common in AI and data science.

Activity: Convert Login Flowchart to Pseudocode

```
START
SET correct_username to "student"
SET correct_password to "python123"
READ username
READ password
IF username equals correct_username AND
   password equals correct_password THEN
    DISPLAY "Login successful"
ELSE
    DISPLAY "Invalid username or password"
END IF
END
```

Note

This is still not Python code. It is a clear logic plan.

From `print()` to Small Programs

In the next activities, we will use only `print()` to create small programs with readable output.

- Each line is still one instruction.
- Python runs the instructions from top to bottom.
- We can use text, spacing, and symbols to make output clearer.
- Even simple programs should be organized and easy to read.

Beginner focus

Before variables, input, and conditions, we can still practice sequence, output, formatting, and careful program layout.

Self-Introduction Program

```
print("Name: Aisha")
print("Field: Computer Science")
print("Goal: Learn Python for AI")
print("Favorite application: Medical image analysis")
```

Exercise variation

Change the values so the program introduces you. Add at least five lines of output.

Formatted Receipt: First Version

```
print("==============================")
print(" PYTHON CAFE")
print("==============================")
print("Item Qty Price")
print("Coffee 2 18.00")
print("Sandwich 1 22.00")
print("------------------------------")
print("Total 40.00")
print("==============================")
```

This uses only output statements, but it introduces layout and formatting thinking.

Formatted Receipt: Using Escape Characters

```
print("Item\tQty\tPrice")
print("Coffee\t2\t18.00")
print("Tea\t1\t8.00")
print("Total\t\t26.00")
```

Useful escape characters

- \n: new line
- \t: tab space
- \\: backslash character
- \": double quote inside a string

Activity 3: Implement Pseudocode Using `print()`

Since Module 1 only introduces `print()`, we can simulate the interaction.

```
print("Stored username: student")
print("Stored password: python123")
print("User enters username: student")
print("User enters password: python123")
print("Checking username and password...")
print("Login successful")
```

Why simulate?

Real input and conditions are introduced in Module 2. For now, focus on sequence and output.

Common Beginner Mistakes

Mistake	Example	Fix
Skipping planning	Starting code without knowing steps	Write algorithm or pseudocode first
Vague instructions	"Process the data"	Specify each action
Confusing syntax and logic	Correct spelling but wrong steps	Desk-check with examples
Ignoring assumptions	Empty list in max algorithm	State assumptions clearly
Forgetting quotes	`print(Hello)`	Use `print("Hello")` for text
Missing parentheses	`print "Hello"`	Use `print("Hello")` in Python 3

Module 3

Variables, Data Types & Input/Output

Variables, types, casting, and user input

Module Content

In this module, we will store and work with data using variables, types, and user input.

- **Variables**: assignment, naming rules, constants.
- **Data types**: `int`, `float`, `complex`, `bool`, `str`, `NoneType`.
- **Type tools**: `type()`, `isinstance()`, and casting.
- **Strings and length**: `len()`, string basics.
- **Input and output**: `input()`, `print()`, and f-strings.
- **Type conversion**: changing types with casting functions.

Learning goal

Store data in the right type, convert between types, and interact with the user through input and output.

Variables: Names for Values

Key idea

A **variable** is a name that refers to a value. The value can change as the program runs.

```
student_name = "Aisha"
age = 20
score = 95.5
age = age + 1
```

- The equal sign = means **assign the value on the right to the name on the left**.
- Python remembers the latest value assigned to a variable.
- A variable name should help humans understand what the value means.

Naming Conventions and Constants

Good variable names

Use **snake_case**: lowercase words separated by underscores.

```
course_name = "Python"
student_count = 35
final_score = 88
```

Avoid

- unclear names: `x`, `data1`
- spaces: `student count`
- starting with a number: `2score`

Constants in Python

Python does not truly lock constants by default. By convention, write values that should not change in uppercase.

```
PI = 3.14159
MAX_ATTEMPTS = 3
```

Primitive Data Types in One Slide

Type	Meaning	Example
`int`	whole number	`students = 40`
`float`	number with decimal point	`price = 19.99`
`complex`	number with real and imaginary parts	`z = 2 + 3j`
`bool`	true/false value	`is_ready = True`
`str`	text data	`name = "Aisha"`
`NoneType`	represents no value yet	`result = None`

Decimal note

For beginner work, `float` is enough. For money or exact decimal arithmetic, Python also has `Decimal` from the `decimal` module.

Checking Types: type() and isinstance()

Key idea

Every value in Python has a type. The type tells Python what operations make sense.

```
age = 20
name = "Aisha"
print(type(age))       # <class 'int'>
print(type(name))      # <class 'str'>
print(isinstance(age, int))   # True
```

- Use `type(value)` when you want to inspect what kind of value you have.
- Use `isinstance(value, type)` when you want a true/false check.

Type Conversion: Changing One Type to Another

Key idea

Type conversion means turning a value from one type into another type.

Explicit casting: you ask for it

```
age_text = "20"
age = int(age_text)
price = float("19.99")
message = str(100)
flag = bool(1)
```

Implicit conversion: Python handles it

```
result = 5 + 2.5
print(result)       # 7.5
print(type(result)) # float
```

Checkpoint

Conversion is useful when input arrives as text, but you need to calculate with numbers.

Strings and `len()`

Key idea

A **string** is text. Strings are written between quotes.

```
course = "Python Fundamentals"
word = "AI"

print(course)
print(len(course))  # number of characters
print(len(word))    # 2
```

- `len(text)` returns how many characters are inside a string.
- Spaces count as characters.
- `len()` also works with other collections later, such as lists.

Printing Values

Normal `print()`

```
name = "Aisha"
score = 95

print(name)
print("Score:", score)
print("A", "B", "C")
```

Formatted strings: `f""`

```
name = "Aisha"
score = 95

print(f"Student: {name}")
print(f"Score: {score}")
print(f"Next score: {score + 1}")
```

Useful options

sep changes the separator between printed values. end changes what happens at the end of the line.

```
print("2026", "05", "03", sep="/")   # 2026/05/03
print("Hello", end=" "); print("World")
```

Getting User Input with input()

Key idea

The **input()** function pauses the program and waits for the user to type something.

```
name = input("Enter your name: ")
print("Hello", name)
```

- The text inside input() is the **prompt** shown to the user.
- Whatever the user types is stored in the variable.
- By default, input() always gives you a **string**.

Beginner checkpoint

Even if the user types 25, Python receives it as "25", not as the number 25.

Input and Type Conversion

When you want to do math with user input, convert it first.

```
age_text = input("Enter your age: ")
age = int(age_text)
print("Next year you will be", age + 1)
```

Useful conversions

```
int("42")      → 42
float("3.5")   → 3.5
str(100)       → "100"
bool(0)        → False
```

Common mistake

```
x = input("Number: ")
print(x + 1)   # error: x is text
```

Rule

Use `input()` to collect text. Use `int()` or `float()` when you need numbers.

Module 4
Operators, Errors & Modules

Operators, error types, and importing modules

Module Content

In this module, we will learn to compute with operators, understand errors, and use modules.

- **Arithmetic operators**: `+`, `-`, `*`, `/`, `//`, `%`, `**`.
- **Comparison operators**: `==`, `!=`, `<`, `>`, `<=`, `>=`.
- **Logical operators**: `and`, `or`, `not`, short-circuit evaluation.
- **Assignment operators**: `+=`, `-=`, and compound forms.
- **Operator precedence**: order of evaluation, parentheses.
- **Error types**: syntax, runtime, and logic errors; reading tracebacks.
- **Debugging**: print-based debugging strategies.
- **Modules**: importing `math` and using built-in functions.

Learning goal

Compute correctly, read error messages confidently, and reuse existing Python tools.

Arithmetic Operators

Key idea

Arithmetic operators perform mathematical calculations on values.

Operator	Meaning	Example
+	addition	5 + 2 gives 7
-	subtraction	5 - 2 gives 3
*	multiplication	5 * 2 gives 10
/	true division	5 / 2 gives 2.5
//	floor division	5 // 2 gives 2
%	remainder	5 % 2 gives 1
**	exponentiation	5 ** 2 gives 25

```
minutes = 130
hours = minutes // 60
remaining = minutes % 60
print(hours, "hours and", remaining, "minutes")
```

Comparison Operators

Key idea

Comparison operators ask a question and produce a Boolean result: `True` or `False`.

Operator	Meaning	Example
`==`	equal to	`score == 100`
`!=`	not equal to	`name != "Ali"`
`<`	less than	`age < 18`
`>`	greater than	`temperature > 40`
`<=`	less than or equal to	`attempts <= 3`
`>=`	greater than or equal to	`grade >= 60`

```
score = 75
print(score >= 60)   # True
```

Important: Use = for assignment. Use == when asking whether two values are equal.

Logical Operators

Key idea

Logical operators combine Boolean expressions.

Operator	Meaning	Example
`and`	true only if both sides are true	`age >= 18 and has_id`
`or`	true if at least one side is true	`is_member or has_coupon`
`not`	reverses the Boolean value	`not is_locked`

```
age = 20
has_id = True
can_enter = age >= 18 and has_id
print(can_enter)
```

Beginner interpretation: Read `age >= 18 and has_id` as: adult and has ID.

Bitwise Operators: A Light Overview

Key idea

Bitwise operators work on the binary representation of integers. Beginners only need a high-level awareness for now.

Operator	Name	Informal meaning
&	bitwise AND	keeps bits that are 1 in both numbers
|	bitwise OR	keeps bits that are 1 in either number
^	bitwise XOR	keeps bits that are different
~	bitwise NOT	flips bits
«	left shift	shifts bits left, often like multiplying by 2
»	right shift	shifts bits right, often like dividing by 2

```
print(5 & 3)   # 1
print(5 | 3)   # 7
```

Where they appear: masks, permissions, embedded systems, and low-level code.

Assignment Operators

Key idea

Assignment operators update variables. They are shortcuts for common update patterns.

Operator	Meaning	Equivalent form
`=`	assign a value	`x = 10`
`+=`	add then assign	`x += 2` means `x = x + 2`
`-=`	subtract then assign	`x -= 2` means `x = x - 2`
`*=`	multiply then assign	`x *= 2` means `x = x * 2`
`/=`	divide then assign	`x /= 2` means `x = x / 2`
`//=`	floor divide then assign	`x //= 2`
`**=`	power then assign	`x **= 2`
`%=`	remainder then assign	`x %= 2`

```
score = 10
score += 5
print(score)   # 15
```

Operator Precedence and Associativity

Key idea

Precedence decides which operator runs first. Associativity decides the direction when operators have the same priority.

Higher first	Examples
Parentheses	`(...)`
Power	`**`
Unary signs	`+x, -x`
Multiply/divide	`* / // %`
Add/subtract	`+ -`
Comparisons	`== < >=`
Logical NOT	`not`
Logical AND	`and`
Logical OR	`or`

```
result = 2 + 3 * 4
print(result)      # 14

result = (2 + 3) * 4
print(result)      # 20
```

Best habit

Use parentheses when they make the expression easier to read, even if Python does not require them.

Short-Circuit Evaluation

Key idea

Python may stop checking a logical expression early if the final answer is already known.

With `and`

If the first part is `False`, Python skips the second part.

```
x = 0
print(x != 0 and 10 / x > 1)
```

No division error happens.

With `or`

If the first part is `True`, Python skips the second part.

```
is_admin = True
print(is_admin or has_permission)
```

The second part is unnecessary.

Why it matters

Short-circuiting helps write safe checks and avoid unnecessary work.

Calling Built-in Functions

Key idea

A **function call** asks Python to run a ready-made tool and return a result.

```
print(abs(-7))              # 7
print(max(4, 9, 2))         # 9
print(min(4, 9, 2))         # 2
print(round(3.14159, 2))    # 3.14
```

- The function name comes first: `abs`, `max`, `min`, `round`.
- Parentheses hold the **arguments**: the values given to the function.
- Some functions need one argument; others can use many.

Beginner checkpoint

A function call has the shape `name(arguments)`. Later, you will learn how to create your own functions.

Types of Errors: The Big Picture

Key idea

Errors are normal. Python uses error messages to tell you what went wrong and where to start looking.

Kind of problem	Meaning	Common Python examples
Syntax error	The code breaks Python grammar rules.	`SyntaxError`
Runtime error	The code starts running, then fails.	`NameError`, `TypeError`, `ValueError`, `ZeroDivisionError`, `IndexError`
Logic error	The code runs, but the answer is wrong.	Wrong formula, wrong condition, wrong order of steps

```
print("Hello"        # SyntaxError: missing closing parenthesis
print(total)         # NameError if total was never created
print(10 / 0)        # ZeroDivisionError
```

Course focus: Do not memorize every error name. Learn to notice the category and read the message calmly.

Reading Tracebacks

A **traceback** is Python's report of an error. It usually tells you the file, line number, error type, and message.

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    age = int("twenty")
ValueError: invalid literal for int() with base 10: 'twenty'
```

How to read it

1. Start near the **bottom**: `ValueError` tells you the type of problem.
2. Look at the **line number**: line 2 is where Python noticed the error.
3. Read the message: `"twenty"` cannot be converted to an integer.

Beginner habit

Do not panic when a traceback looks long. Find the last line, then check the line of code it points to.

Print-Based Debugging

Key idea

Debugging means finding and fixing mistakes. The simplest debugging tool is `print()`.

```
price = 50
quantity = 3
print("DEBUG price:", price)
print("DEBUG quantity:", quantity)

total = price * quantity
print("DEBUG total:", total)
```

- Print important variables before and after a calculation.
- Use clear labels like `"DEBUG total:"`.
- Remove or comment out debug prints after fixing the problem.

When it helps: checking variable values, sequence, and whether a line actually runs.

Python Modules: Reusing Existing Tools

Key idea

A **module** is a file or library that contains Python code we can reuse instead of writing everything ourselves.

```
import math

print(math.sqrt(25))
print(math.pi)
```

- Python already includes many useful modules.
- Use `import` to bring a module into your program.
- Use `module.function()` to call tools from that module.

Beginner interpretation

`math.sqrt(25)` means: use the `sqrt` function that lives inside the `math` module.

The math Library: A First Example

The math module gives us common mathematical functions and constants.

```
import math

radius = 3
area = math.pi * radius ** 2

print(area)
print(round(area, 2))
```

Examples you may see

`math.pi`	π value
`math.sqrt(x)`	square root
`math.floor(x)`	round down
`math.ceil(x)`	round up
`math.sin(x)`	sine function

Important note

Some simple functions, such as `round()`, `min()`, `max()`, and `abs()`, are built in. You can use them without importing `math`.

Module 5
Conditionals & Loops

If/elif/else decisions and for/while repetition

Module Content

In this module, we will write programs that make decisions and repeat steps.

- **Conditionals**: `if`, `elif`, `else`; nested conditionals.
- **Truthiness and falsiness**: what Python treats as true or false.
- **Ternary expressions**: one-line conditional values.
- **Input validation**: checking user input with `if`.
- **For loops**: iterating over sequences and `range()`.
- **While loops**: condition-controlled repetition.
- **Loop control**: `break`, `continue`, `pass`, `else`.
- **Loop patterns**: accumulation, search, nested loops, anti-patterns.

Learning goal

Write programs that choose paths and repeat work — the two most powerful tools in programming.

Why Programs Need Decisions

Key idea

A program often needs to choose an action based on data, input, or a situation.

- Is the temperature safe?
- Did the user enter a valid number?
- Is the score high enough to pass?
- Should the program show an error message?

```
temperature = 38

if temperature > 37.5:
    print("Fever warning")
```

Programming lesson

An `if` statement is how we turn a question into a decision the computer can execute.

The Basic `if` Structure

A simple `if` statement runs a block only when the condition is true.

```
floor = 20
actual_floor = floor

if floor > 13:
    actual_floor = floor - 1

print(actual_floor)
```

- The header ends with a colon `:`.
- The indented lines are the block controlled by the `if`.
- If the condition is false, the block is skipped.

Beginner habit

Read the condition aloud: "if floor is greater than 13, subtract 1 from it."

Choosing Between Two Paths: `if` and `else`

Key idea

Use `else` when the program should do one thing if the condition is true and another thing otherwise.

```
floor = int(input("Floor: "))

if floor > 13:
    actual_floor = floor - 1
else:
    actual_floor = floor

print("Actual floor:", actual_floor)
```

Flow of control

Only one branch runs: either the `if` branch or the `else` branch.

Multiple Alternatives: `if`, `elif`, `else`

When there are more than two possible outcomes, use `elif`.

```
score = 82

if score >= 90:
    print("Excellent")
elif score >= 70:
    print("Passed")
elif score >= 50:
    print("Needs practice")
else:
    print("Try again")
```

- Python checks the branches from top to bottom.
- As soon as one condition is true, that branch runs.
- The remaining branches are skipped.

Order Matters in `elif` Chains

Key idea

Put the more specific or larger cutoff conditions first when using `elif`.

Correct order

```
richter = 7.1

if richter >= 8.0:
    print("Severe")
elif richter >= 7.0:
    print("Major")
elif richter >= 4.5:
    print("Light")
```

Wrong order

```
richter = 7.1

if richter >= 4.5:
    print("Light")
elif richter >= 7.0:
    print("Major")
```

What happens? In the wrong order, `7.1 >= 4.5` is already true, so Python never reaches the stronger category.

Nested Conditionals and Readability

A **nested conditional** is an `if` statement inside another `if` statement.

Nested version

```
if age >= 18:
    if has_id:
        print("Allowed")
    else:
        print("Need ID")
else:
    print("Too young")
```

Flatter version

```
if age < 18:
    print("Too young")
elif not has_id:
    print("Need ID")
else:
    print("Allowed")
```

Beginner guideline

Nested conditionals are useful, but too many levels make code hard to read. Prefer simple branches when possible.

Truthiness and Falsiness

Key idea

In Python, some values behave like `False` when used in a condition. This is called **falsiness**.

Common falsy values

- `False`
- `None`
- `0`, `0.0`
- `""` empty string
- `[]` empty list

```
name = ""

if name:
    print("Name exists")
else:
    print("No name entered")
```

Rule of thumb

Empty or missing values are usually falsy. Non-empty values are usually truthy.

Ternary Expressions: Small Decisions in One Line

Key idea

A **ternary expression** chooses between two values in one line.

```
age = 20
status = "adult" if age >= 18 else "minor"
print(status)
```

The general form is:

```
value_if_true if condition else value_if_false
```

When to use it

Use it for short, simple choices. If the condition becomes long or confusing, use a normal `if/else` block instead.

Input Validation with if

Key idea

Input validation means checking user input before trusting it in calculations or decisions.

```
floor = int(input("Floor: "))

if floor == 13:
    print("Error: there is no 13th floor.")
elif floor <= 0 or floor > 20:
    print("Error: floor must be between 1 and 20.")
else:
    actual_floor = floor
    if floor > 13:
        actual_floor = floor - 1
    print("Actual floor:", actual_floor)
```

Beginner takeaway

Use conditions to protect your program from invalid cases before running the main logic.

Loops: Repeating Work

Key idea

A **loop** repeats a block of code. We use loops when the same idea must happen many times.

- Process every item in a list.
- Print numbers from 1 to 10.
- Keep asking until input is valid.
- Repeat calculations until a goal is reached.

```
for number in [1, 2, 3]:
    print(number)
```

Beginner interpretation

Do the indented instruction once for each value.

The for Loop: Iterating Over Values

Use a for loop when you already have a sequence, range, or iterable to go through.

```
fruits = ["apple", "banana", "orange"]

for fruit in fruits:
    print(fruit)
```

Structure

`for variable in iterable:` followed by an indented block.

- The variable receives one value at a time.
- The loop body runs once per value.
- This works with lists, strings, ranges, and many other iterable objects.

The range() Function: start, stop, step

range() creates a sequence of integers, often used for count-controlled loops.

```
for i in range(5):
    print(i)      # 0, 1, 2, 3, 4

for i in range(1, 6):
    print(i)      # 1, 2, 3, 4, 5

for i in range(1, 10, 2):
    print(i)      # 1, 3, 5, 7, 9
```

Remember

`range(stop)`	starts at 0
`range(start, stop)`	stops before stop
`range(start, stop, step)`	jumps by step

Common trap

The stop value is **not included**. This is a common source of off-by-one errors.

The `while` Loop: Condition-Controlled Repetition

Use a `while` loop when repetition depends on a condition being true.

```
balance = 10000
year = 0

target = 20000
rate = 0.05

while balance < target:
    year = year + 1
    balance = balance + balance * rate

print("Years:", year)
```

Loop checklist

Initialize something before the loop, test a condition, and update something inside the loop so the condition can eventually become false.

`for` vs. `while`: Choosing the Right Loop

Loop	Best for	Example idea
`for`	A known collection or count	For each student, print the grade.
`for` **with** `range()`	Repeating a known number of times	Print numbers from 1 to 10.
`while`	Repeating until a condition changes	Keep asking until the password is correct.

```
for i in range(3):
    print("Try", i)
```

```
password = ""
while password != "python":
    password = input("Password: ")
```

Loop Control: break, continue, and pass

These keywords change what happens inside a loop.

break

Exit the loop early.

```
for n in nums:
    if n < 0:
        break
```

continue

Skip to the next iteration.

```
for n in nums:
    if n == 0:
        continue
    print(n)
```

pass

Do nothing for now.

```
for n in nums:
    pass
```

Beginner advice

Use these sparingly. A clear loop is usually better than a clever loop.

The else Clause on Loops

Key idea

A loop `else` runs only if the loop finishes normally without hitting `break`.

```
numbers = [2, 4, 6, 8]

for n in numbers:
    if n < 0:
        print("Found a negative number")
        break
else:
    print("No negative numbers found")
```

Useful pattern

Loop `else` is useful for search problems: *if you never found the item, do this final action.*

Nested Loops: Loops Inside Loops

A **nested loop** has one loop inside another loop. This is useful for tables, grids, and pairwise comparisons.

```
for row in range(1, 4):
    for col in range(1, 4):
        print(row, col)
```

Use cases

- Printing tables
- Comparing every pair
- Working with rows and columns

Performance awareness

If the outer loop runs 100 times and the inner loop runs 100 times, the body runs 10,000 times.

Common Loop Algorithms

Many programs repeat the same loop patterns with different data.

```
values = [3, -2, 7, -1]

total = 0
negatives = 0

for value in values:
    total += value
    if value < 0:
        negatives += 1

print(total)
print(negatives)
```

Patterns to recognize

- Sum values
- Count matches
- Search for a value
- Find minimum or maximum
- Build a new list

Checkpoint

Before coding, describe the pattern in words: "go through every value and update something."

Avoiding Loop Anti-Patterns

Common mistake

Do not modify a list while looping over the same list. It can skip values or produce confusing behavior.

Risky

```
numbers = [1, 2, 3, 4]

for n in numbers:
    if n % 2 == 0:
        numbers.remove(n)
```

Safer

```
numbers = [1, 2, 3, 4]

odds = []
for n in numbers:
    if n % 2 != 0:
        odds.append(n)
```

Rule of thumb

If you need a changed version of a list, build a new list instead of editing the old one while looping.

Loop Thinking: Initialization, Condition, Update

Key idea

Every loop has a rhythm: setup, repeat, update, stop.

```
count = 1         # initialize

while count <= 5: # condition
    print(count)
    count += 1    # update
```

- What value do we start with?
- When should the loop continue?
- What changes each repetition?
- How do we know it will stop?

Beginner habit

Trace a small example by hand. Most loop bugs become easier to see when you write down each step.

Thank you!

Questions and discussion

Course takeaway

Python is a tool for clear thinking. Start with an algorithm, express it in code, test it, and improve it.